# SecureLens Security Audit Report

Generated on 2026-07-23 17:59:17 UTC | Powered by SecureLens Engine & Gemini Cascade RAG

## Executive Summary & Project Statistics

| | | | |
|---|---|---|---|
| **Project Target** | DemoPortal | **Overall Risk Score** | **HIGH** |
| **Files Scanned** | 2 | **Critical Findings** | 0 |
| **Lines Scanned** | 15 | **High Findings** | 2 |
| **Analysis Duration** | 24134 ms | **Medium/Low Findings** | 1 |

## Vulnerabilities Overview Table

| File / Line | Severity | Vulnerability | Standard |
|---|---|---|---|
| app.py:6 | **HIGH** | Hardcoded Secret | CWE-798 \| A07:2021 |
| app.py:9 | **MEDIUM** | Weak Crypto | CWE-327 \| A02:2021 |
| database.py:4 | **HIGH** | Sql Injection | CWE-89 \| A03:2021 |

# Detailed Security Findings & Remediation Guidance

## Finding 1: Hardcoded Secret

**Location**: app.py:6 | **Severity**: **HIGH** | **Classification**: CWE-798 (A07:2021)

**Description & Vulnerability Analysis:**

Hardcoding secrets like a `SECRET_KEY` directly into the source code is a dangerous practice because anyone who gains read access to the codebase—whether through public source control, compromised developer workstations, or decompiled binaries—can instantly extract the sensitive value. Hardcoded secrets cannot be easily rotated without modifying and redeploying the code. The recommended fix is to externalize the secret by retrieving it from environment variables using `os.environ` or an external secret management service, ensuring the secret is kept out of version control entirely.

**Attack Vector Scenario:**

An attacker gains unauthorized read access to the project's public GitHub repository or a compromised deployment server containing the source code. They locate the hardcoded `SECRET_KEY` on line 6 and use it to forge cryptographic session tokens or bypass authentication mechanisms, granting themselves administrative access to the application.

| Vulnerable Code Snippet | Recommended Secure Fix |
| --- | --- |
| `SECRET_KEY = '********'` | `import os`<br><br>`SECRET_KEY = os.environ.get('SECRET_KEY')` |

**Citations & References**: OWASP Top 10 - Identification and Authentication Failures (via ChromaDB)

## Finding 2: Weak Crypto

**Location**: app.py:9 | **Severity**: **MEDIUM** | **Classification**: CWE-327 (A02:2021)

**Description & Vulnerability Analysis:**

The code uses the MD5 cryptographic hash function via hashlib.md5(). MD5 is cryptographically broken and vulnerable to collision attacks, meaning an attacker can generate two different inputs that produce the same hash output. Replacing it with a modern, secure hashing algorithm like SHA-256 (hashlib.sha256()) ensures cryptographic integrity and prevents collision-based exploits.

**Attack Vector Scenario:**

An attacker intercepts data or exploits a system relying on MD5 for digital signatures or integrity checks. By crafting a malicious file or input that produces the same MD5 hash as a legitimate one (a collision), the attacker bypasses integrity controls and tricks the application into accepting unauthorized or malicious data.

| Vulnerable Code Snippet | Recommended Secure Fix |
| --- | --- |
| `hashlib.md5()` | `hashlib.sha256()` |

**Citations & References**: OWASP Reference

## Finding 3: Sql Injection

**Location**: database.py:4 | **Severity**: **HIGH** | **Classification**: CWE-89 (A03:2021)

**Description & Vulnerability Analysis:**

SQL Injection (SQLi) occurs when untrusted user input is directly concatenated or interpolated into a database query string instead of being passed safely. This allows an attacker to manipulate the query structure, potentially gaining unauthorized access to sensitive data, modifying records, or executing administrative operations. The fix replaces string formatting or concatenation with parameterized queries, ensuring the database engine treats user input strictly as data rather than executable code.

**Attack Vector Scenario:**

An attacker inputs a malicious payload containing SQL commands (such as ' OR '1'='1) into a vulnerable input field. Because the input is directly concatenated into the query string, the database interprets the payload as part of the query logic, bypassing authentication and returning unauthorized records.

| Vulnerable Code Snippet | Recommended Secure Fix |
| --- | --- |
| `db_conn.execute(query)` | `db_conn.execute("SELECT * FROM users WHERE usernam e = %s AND password = %s", (user, password))` |

**Citations & References**: OWASP Top 10 - SQL Injection Prevention (via ChromaDB)